漫步数码时代，

铸就精彩人生，

程序员修炼之路，

从精雕算法开始……

第3章 动态规划

（Dynamic Programming）

（二）

主讲人：周正深

3.3.1 投资问题

例3.4 m元钱，n项投资，$f_i(x)$表示将x元投入第i个项目的效益。

目标函数 $\max\{ f_1(x_1) + f_2(x_2) + \cdots + f_n(x_n)\}$

约束条件 $x_1 + x_2 + \cdots + x_i + \cdots + x_n = m,\ x_i \in N$

实例：5万元钱，4个项目，效益函数如下表所示

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

蛮力算法：

找出所有投资方案，计算出每个方案的效益，再找出最优。

第1个1元投资n个项目，有n 种方案。

第2个1元投资n个项目，有n 种方案。

……

第m个1元投资n个项目，有n种方案。

总共有n^m个方案，每种方案求效益需要$O(m)$次加法。

所以，加法运算复杂度$O(nn^m)$，比较运算复杂度$O(n^m)$

动态规划算法：

1. 建模

问题的解是向量$\langle x_1, x_2, \cdots, x_i, \cdots, x_n \rangle$，$x_i$是投给项目$i$的钱数，$i=1,2,\cdots,n$

目标函数 $\max\{f_1(x_1) + f_2(x_2) + \cdots + f_n(x_n)\}$

约束条件 $x_1 + x_2 + \cdots + x_n = m$，$x_i \in N$

2. 子问题划分和计算顺序

①子问题的界定：由参数k和x界定

k：对前k个项目投资 1, 2，…，n；k不超过n

x：投资的总钱数 1, 2，…，m； x不超过m

这两个参数与矩阵相乘问题的参数有什么区别？

原始输入： $k = n$， $x = m$

项目个数n对应矩阵个数n，投资金额m对应括号位置，都是影响问题规模的参数，但括号位置数量由矩阵个数决定，实质上是一个参数。所以前者复杂度$O(nm^2)$后者$O(n^3)$

②子问题计算顺序

先，k=1, 2, …, n

再，对于给定的k，x=1, 2, …, m

3. 优化函数的递推方程

设$F_k(x)$表示x 元钱投给前k 个项目的最大效益， $x_k(x)$为对应$F_k(x)$时应分给第 k 个项目的钱数.

多步判断：假设知道p 元钱（$p \le x$）投给前k–1个项目的最大效益$F_{k-1}(p)$，确定x 元钱投给前k 个项目的分配方案

递推方程和边界条件：

$$F_k(x) = \max_{0 \le x_k \le x} \{ f_k(x_k) + F_{k-1}(x - x_k) \} \quad k > 1$$

$$F_1(x) = f_1(x) \qquad k = 1$$

4. 求解

（1）计算过程

k=1时，实例的计算

x	$f_1(x)$	$f_2(x)$	$f_3(x)$	$f_4(x)$
0	0	0	0	0
1	11	0	2	20
2	12	5	10	21
3	13	10	30	22
4	14	15	32	23
5	15	20	40	24

k=1为初值

$F_1(1)=11$，$F_1(2)=12$，$F_1(3)=13$，$F_1(4)=14$，$F_1(5)=15$

k=2时，实例的计算

x	$f_1(x)$	$f_2(x)$
0	0	0
1	11	0
2	12	5
3	13	10
4	14	15
5	15	20

投1万方案：(0, 1) (1, 0)

$F_2(1)=\max\{f_2(1), F_1(1)\}=11$， → $x_2(1)=0$

投2万方案：(0, 2) (1, 1) (2, 0)

$F_2(2)=\max\{f_2(2), F_1(1)+f_2(1), F_1(2)\}=12$， $x_2(2)=0$

投3万方案：(0, 3) (1, 2) (2, 1) (3, 0)

$F_2(3)=\max\{f_2(3), F_1(1)+f_2(2),$

$F_1(2)+f_2(1), F_1(3)\}=16$， $x_2(3)=2$

投4万方案：(0, 4) (1, 3) (2, 2) (3, 1) (4, 0)

$F_2(4)=\max\{f_2(4), F_1(1)+f_2(3), F_1(2)+f_2(2), F_1(3)+f_2(1), F_1(4)\}=21$， $x_2(4)=3$

投5万方案：(0, 5) (1, 4) (2, 3) (3, 2) (4, 1) (5, 0)

$F_2(5)=\max\{f_2(5), F_1(1)+f_2(4), F_1(2)+f_2(3), F_1(3)+f_2(2),$

$F_1(4)+f_2(1), F_1(5)\}=26$， → $x_2(5)=4$

实例计算汇总　　备忘录和解

x	$F_1(x)$	$x_1(x)$	$F_2(x)$	$x_2(x)$	$F_3(x)$	$x_3(x)$	$F_4(x)$	$x_4(x)$
1	11	1	11	0	11	0	20	1
2	12	2	12	0	13	1	31	1
3	13	3	16	2	30	3	33	1
4	14	4	21	3	41	3	50	1
5	15	5	26	4	43	4	61	1

$F_k(x)$表示x 元钱投给前k 个项目的最大效益， $x_k(x)$为标记函数，即$F_k(x)$ 取得最大值时应分给第 k 个项目的钱数.

最优解$F_4(5)=61$

(2)解的追踪

x	$F_1(x)$	$x_1(x)$	$F_2(x)$	$x_2(x)$	$F_3(x)$	$x_3(x)$	$F_4(x)$	$x_4(x)$
1	11	1	11	0	11	0	20	1
2	12	2	12	0	13	1	31	1
3	13	3	16	2	30	3	33	1
4	14	4	21	3	41	3	50	1
5	15	5	26	4	43	4	61	1

$x_4(5)=1$ ➔ $x_4=1$, $x_3(5-1)=x_3(4)$

$x_3(4)=3$ ➔ $x_3=3$, $x_2(4-3)=x_2(1)$

$x_2(1)=0$ ➔ $x_2=0$, $x_1(1-0)=x_1(1)$

$x_1(1)=1$ ➔ $x_1=1$

输出解为： $x_1=1$，$x_2=0$，$x_3=3$，$x_4=1$，$F_4(x)=61$

投资额在前k个和第k个项目两者之间分摊$O(m)$种情况

5. 算法的复杂度分析：

$$F_k(x) = \max_{0 \le x_k \le x}\{f_k(x_k) + F_{k-1}(x - x_k)\} \quad k > 1$$

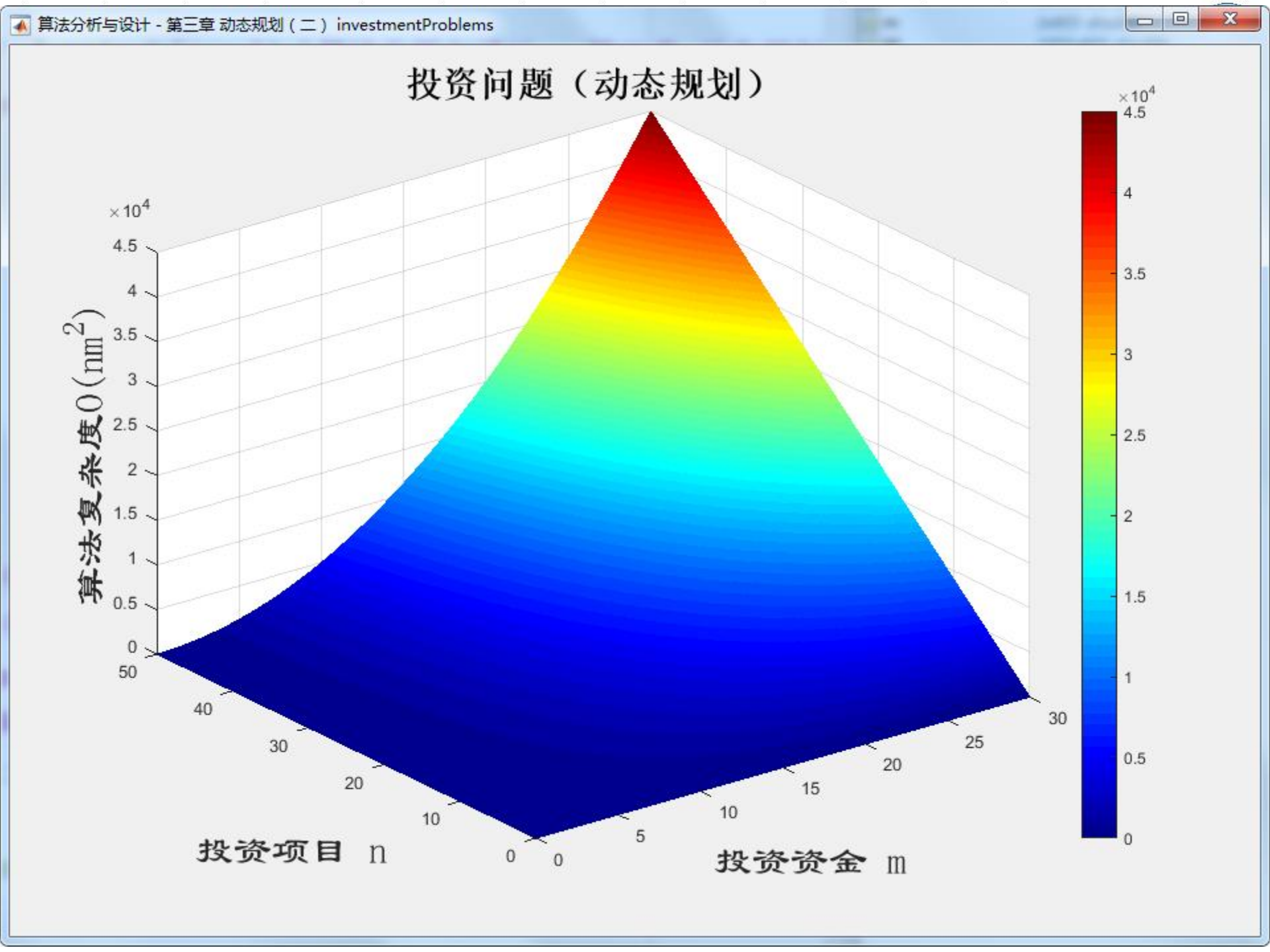
$$F_1(x) = f_1(x)$$

对第 $F_k(x)$ 项 ($2 \le k \le n$, $1 \le x \le m$) 要 x+1 次加法, x 次比较

∴ $F_k(x)$ 的算法复杂度为$O(m)$

∵ 备忘录表中有 m 行 n 列，共计 nm 项

∴ 本算法的时间复杂度$O(nm^2)$


算法分析与设计 - 第三章 动态规划（二） investmentProblems
投资问题（动态规划）
算法复杂度O(nm^2)
×10^4
4.5
4
3.5
3
2.5
2
1.5
1
0.5
0
50
40
30
20
10
0
投资项目 n
0
5
10
15
20
25
30
投资资金 m

投资问题小结

投资问题的动态规划算法

1. 用两个不同类型的参数界定子问题

2. 确定子问题计算顺序

3. 列出优化函数的递推方程和初值

4. 最优解的计算及追踪

5. 时间复杂度分析： $O(nm^2)$

投资问题小结

投资问题的动态规划算法

1. 用两个不同类型的参数界定子问题

2. 确定子问题计算顺序

3. 列出优化函数的递推方程和初值

4. 最优解的计算及追踪

5. 时间复杂度分析： $O(nm^2)$

3.3.2 完全背包问题

例3.5 一个旅行者随身携带一个背包。可以放入背包的物品有n种，每种物品的重量和价值分别为w_i、v_i。如果背包的最大重量限制是b，每种物品可以放多个，怎样选择放入背包的物品以使得背包的价值最大？不妨设上述w_i，v_i，b都是正整数。

实例：n=4，b=10

v_1=1，v_2=3，v_3=5，v_4=9

$w1$=2，w_2=3，w_3=4，w_4=7

蛮力算法：

找出所有装包方案，计算出每个方案的价值，再找出最优。

类似于投资问题。下面的b用m代替

所以，加法运算复杂度$O(mn^m)$，比较运算复杂度$O(n^m)$

动态规划算法：

1. 建模

解是$\langle x_1, x_2, \cdots, x_i, \cdots, x_n \rangle$，其中$x_i$是装入背包的第$i$种物品个数。

目标函数 $\max \sum_{i=1}^{n} v_i x_i$

约束条件 $\sum_{i=1}^{n} w_i x_i \le b,\quad x_i \in N$(非负整数)

线性规划问题：由线性条件约束的线性函数取得最大或最小的问题。

整数规划问题：线性规划问题的变量x_i都是非负整数。

2. 子问题划分和计算顺序

①子问题的界定：由参数k和y界定

k：只有前k种物品 1, 2，…，n。　　k不超过n

y：背包的总重量　1, 2，…，b。　　y 不超过b。

原始输入：　$k=n$，$y=b$

②子问题计算顺序

$k=1, 2, \cdots, n$

对于给定的k，分别求：$y=1, 2, \cdots, b$时的最大价值。

3. (1) 优化函数的递推方程

全装前k-1种物品,或第k种物品至少装1个，共两种情况

设$F_k(y)$:装前k 种物品总重量不超过y 背包达到的最大价值

$$F_k(y)=\max\{F_{k-1}(y),\ F_k(y-w_k)+v_k\}$$

$$F_0(y)=0,\quad 0\le y\le b,\quad F_k(0)=0,\quad 0\le k\le n$$

$$F_1(y)=\left\lfloor\frac{y}{w_1}\right\rfloor v_1,\quad F_k(y)=-\infty,\quad y<0$$

上例公式，上例中同种项目投资额不同则收益不同，不能用第二种递推公式。

类似于投资问题，如何写递推方程？这两种写法有什么区别？

$$F_k(y)=\max_{0\le x_k\le\lfloor y/w_k\rfloor}\{F_{k-1}(y-x_kw_k)+x_kv_k\}$$ 复杂度$O(b)$

$$F_k(y)=\max\{F_{k-1}(y),\ F_k(y-w_k)+v_k\}$$ 因$F_k(y-w_k)$值已有，故为常数时间

(2)标记函数：

$i_k(y)$：装前k 种物品总重量不超过y 背包达到最大价值时，装入物品的最大标号

$$i_k(y)=\begin{cases} i_{k\text{-}1}(y) & F_{k-1}(y)>F_k(y-w_k)+v_k \\ k & F_{k-1}(y)\le F_k(y-w_k)+v_k \end{cases}$$

$$i_1(y)=\begin{cases} 0 & y<w_1 \\ 1 & y\ge w_1 \end{cases}$$

4. 求解

(1) 计算：对于实例 $n=4$，$b=10$

$v_1=1$，$v_2=3$，$v_3=5$，$v_4=9$

$w_1=2$，$w_2=3$，$w_3=4$，$w_4=7$

$F_k(y)$ 的表如下：

k \ y	1	2	3	4	5	6	7	8	9	10
1	0	1	1	2	2	3	3	4	4	5
2	0	1	3	3	4	6	6	7	9	9
3	0	1	3	5	5	6	8	10	10	11
4	0	1	3	5	5	6	9	10	10	12

计算标记函数

$i_k(y)$的表如下：

k \ y	1	2	3	4	5	6	7	8	9	10
1	0	1	1	1	1	1	1	1	1	1
2	0	1	2	2	2	2	2	2	2	2
3	0	1	2	3	3	3	3	3	3	3
4	0	1	2	3	3	3	4	3	4	4

最优解：$F_4(10)=12$， $i_4(10)=4$

(2) 追踪解：

$i_k(y)$ 的表如下：

k \ y	1	2	3	4	5	6	7	8	9	10
1	0	1	1	1	1	1	1	1	1	1
2	0	1	2	2	2	2	2	2	2	2
3	0	1	2	3	3	3	3	3	3	3
4	0	1	2	3	3	3	4	3	4	4

$i_4(10)=4 \rightarrow x_4 \geqslant 1$

$i_4(10-w_4) = i_4(10-7) = i_4(3)=2 \rightarrow x_2 \geqslant 1$，$x_4=1$，$x_3=0$

$i_2(3-w_2) = i_2(3-3) = i_2(0) = 0 \rightarrow x_2=1$，$x_1=0$

解　$x_1=0$，$x_2=1$，$x_3=0$，$x_4=1$，价值12

(3)追踪算法TrackSolution　　算法复杂度$O(b)$

输入：　$i_k(y)$表，k=1，2，…，n，y=1，2，…，b

输出：x_1，x_2，，x_n，n种物品的装入量

1. for $k \leftarrow 1$ to n do $x_k \leftarrow 0$)　//初始化
2. $y \leftarrow b$ ，$k \leftarrow n$
3. $k \leftarrow i_k(y)$　　//初始追踪位置
4. $x_k \leftarrow 1$
5. $y \leftarrow y - w_k$
6. while $i_k(y)==k$ do　　//继续放第k种物品(同种物品)
7. 　　$y \leftarrow y - w_k$
8. 　　$x_k \leftarrow x_k + 1$
9. if $i_k(y) \neq 0$ then goto 3 //继续追下一种

5. 时间复杂度分析

根据公式：

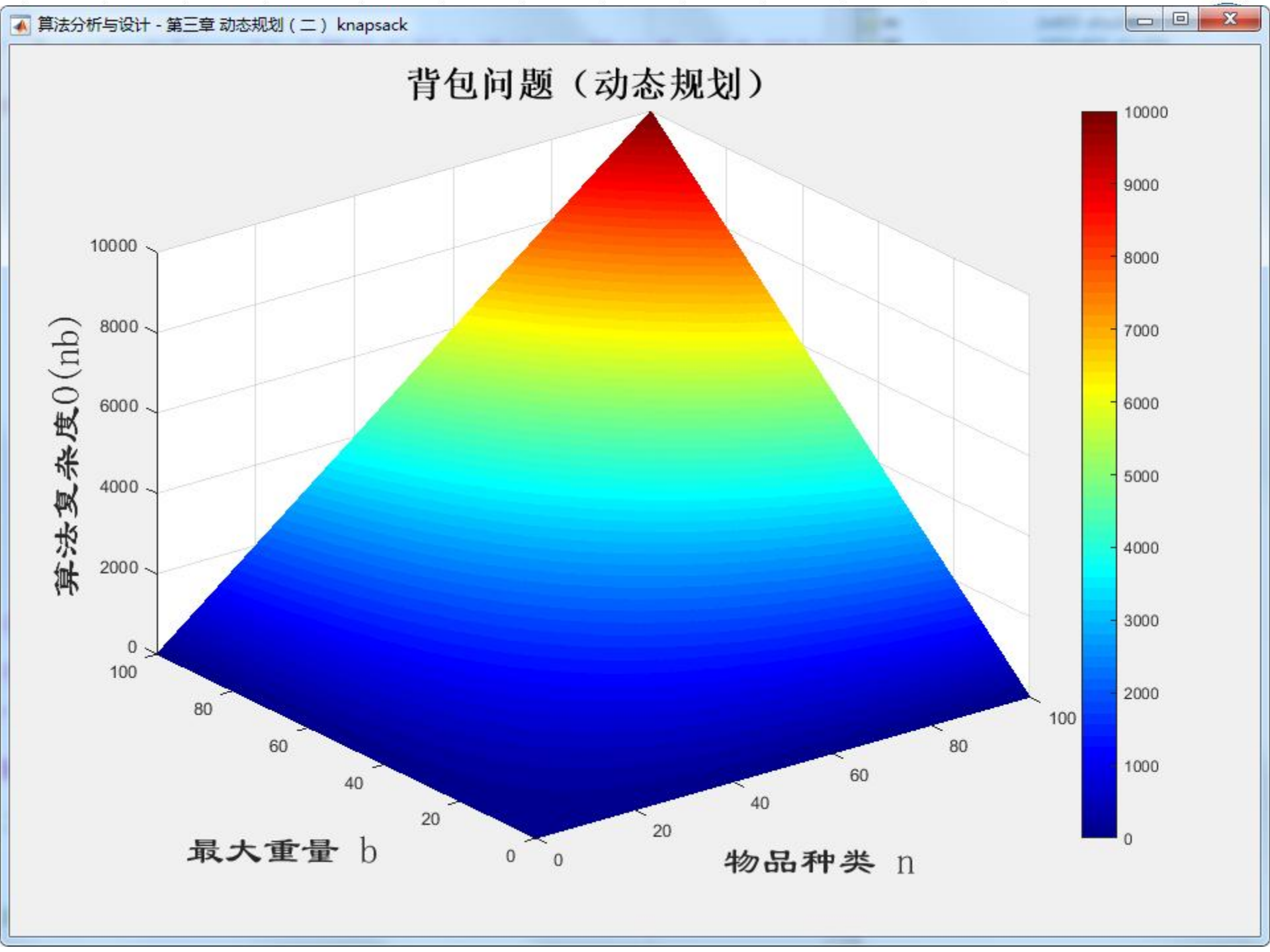
$$F_k(y) = \max\{F_{k-1}(y),\ F_k(y - w_k) + v_k\}$$

备忘录需计算nb项，每项常数时间，计算得

时间复杂度$O(nb)$

伪多项式时间算法：所需时间为参数b和n的多项式，不是输入规模的多项式。

参数b是整数，表达b需要$\log b$位，输入规模以n和$\log b$为参数。


算法分析与设计 - 第三章 动态规划（二） knapsack
背包问题（动态规划）
算法复杂度O(nb)
10000
8000
6000
4000
2000
0
100
80
60
40
20
0
最大重量 b
0
20
40
60
80
100
物品种类 n
10000
9000
8000
7000
6000
5000
4000
3000
2000
1000
0

背包问题总结

背包问题：

1. 背包问题的子问题界定

2. 列优化函数的方程和边界条件

3. 自底向上计算，设计备忘录(表格)

4. 标记函数的设立和追踪算法

5. 算法时间复杂度分析$O(bn)$

6. 背包问题的推广及应用(见下页)

6. 背包问题的推广

①物品数受限背包　第i种物品最多用n_i个。

②0-1背包问题　$x_i = 0, 1$　　$i=0, 1, 2, \cdots, n$。

③多背包问题　m个背包, 背包j装入最大重量B_j，$j=1, 2, \cdots, m$。在满足所有背包重量约束的条件下，使装入物品价值最大。

④二维背包问题　每件物品有重量w_i和体积t_i, $i=1, 2, \cdots, n$。背包总重不超过b, 体积不超过V, 如何选择物品以得到最大价值。

习题3 P83 3.1

3.1 用动态规划算法求解下列组合优化问题

$$\max\{g_1(x_1)+g_2(x_2)+g_3(x_3)\}$$

$$x_1^2+x_2^2+x_3^2\le 10$$

$$x_1,\ x_2,\ x_3\text{为非负整数}$$

其中函数$g_1(x)$，$g_2(x)$，$g_3(x)$的值如下表：

x	$g_1(x)$	$g_2(x)$	$g_3(x)$	x	$g_1(x)$	$g_2(x)$	$g_3(x)$
0	2	5	8	2	7	16	17
1	4	10	12	3	11	20	22

结 束

谢谢收看，再见！